

Descripción de la Persistencia

Resumen

El sistema utiliza **PostgreSQL 14** en la nube a través de **Neon.tech** como motor de base de datos principal, con una base de datos compartida que contiene **3 esquemas** independientes, uno por cada microservicio. Adicionalmente, el **BFF** utiliza **H2 en memoria** para datos de autenticación y tickets. Las APIs externas (clima e indicadores económicos) se consultan bajo demanda sin persistencia local.

1. Base de Datos Principal — PostgreSQL (Neon.tech)

- **Host:** Neon.tech (AWS us-east-2)
- **Puerto:** 5432
- **Motor:** PostgreSQL 14
- **Esquemas:** `ventas` , `datos_org` , `indicadores`

1.1 Schema: `ventas` (`ms-ventas`)

Tabla	Descripción
<code>venta</code>	Cabecera de venta con fecha, sucursal, total
<code>detalle_venta</code>	Líneas de detalle con producto, cantidad, precio
<code>producto</code>	Catálogo de productos con precio, stock, categoría
<code>sucursal</code>	Sucursales con ubicación y datos de contacto

Relaciones:

- `venta` 1:N → `detalle_venta`
- `detalle_venta` N:1 → `producto`
- `venta` N:1 → `sucursal`

1.2 Schema: `datos_org` (`ms-datos-org`)

Tabla	Descripción
<code>empleado</code>	Empleados con rol, sucursal, datos personales
<code>departamento</code>	Departamentos de la organización

Relaciones:

- `empleado` N:1 → `departamento`

1.3 Schema: `indicadores` (`ms-indicadores`)

Tabla	Descripción
<code>indicador</code>	Catálogo de indicadores KPI
<code>valor_indicador</code>	Valores históricos de cada indicador

categoria_indicador	Categorías de indicadores
---------------------	---------------------------

Relaciones:

- indicador N:1 → categoria_indicador
- valor_indicador N:1 → indicador

2. Base de Datos Secundaria — H2 (BFF)

El **BFF** utiliza H2 en modo embebido (archivo `./data/bff`) para datos que no requieren la escalabilidad de PostgreSQL:

Tabla	Descripción
usuario	Usuarios del sistema con credenciales y roles
ticket	Tickets de soporte y consultas

Motivación: Separar la autenticación y la gestión de tickets del dominio transaccional de ventas, permitiendo que el BFF opere de forma independiente incluso si PostgreSQL no está disponible.

3. APIs Externas (Sin Persistencia Local)

API	Endpoint	Uso
OpenWeather	api.openweathermap.org	Datos climáticos para reportes
Mindicador.cl	mindicador.cl/api	Indicadores económicos (UF, USD, IPC)

Estos datos se consultan bajo demanda y se cachean en memoria (Caffeine) por 5 minutos.

4. Configuración de Conexión

Variables de Entorno (`.env`)

```
DB_HOST=ep-tu-proyecto.us-east-2.aws.neon.tech
DB_PORT=5432
DB_NAME=neondb
DB_USER=tu_usuario
DB_PASSWORD=tu_password
```

JDBC URL por Esquema

```
jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}?currentSchema=ventas
jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}?currentSchema=datos_org
jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}?currentSchema=indicadores
```

5. Estrategia de Inicialización

Cada microservicio ejecuta un `DataInitializer` al arrancar que:

1. Verifica si los datos semilla ya existen
 2. Si no existen, inserta datos de prueba
 3. Los seed scripts `seed_db.py` y `seed_ventas.py` permiten recargar datos manualmente
-

6. Stored Procedures

El archivo `docs/stored-procedures.sql` contiene funciones y procedimientos almacenados para:

- Cálculo de resúmenes por período
- Reportes de ventas por sucursal
- Indicadores de rendimiento agregados